

Azure Machine Learning

 Azure ML Designer

 데이터 정보(변수 타입, 결측치 존재 여부, 클래스 불균형 여부 등)

 분석 목적 및 후보 모델 선정

 모델 평가 취합

군집 모델

K-means Clustering

K-means Clustering + 로지스틱 회귀

분류 모델

랜덤 포레스트 (Two-Class Decision Forest)

Two-Class Random Forest

Two-Class Boosted Decision Tree + 튜닝

Two-Class Decision Forest + 카이제곱 피처 선택

Two-Class Support Vector Machine (SVM)

Two-Class Logistic Regression + Stratified Split

Two-Class Logistic Regression + Cross Validation

Boosted Decision Tree + SMOTE

회귀 모델

선형 회귀 (Linear Regression)

Decision Tree Regression

시각화 Code

Heatmap

Scatterplot

Normalize Transformation Method 정리

 모델별 추천 방식

군집 모델

K-means Clustering

- 데이터 포인트를 유사도 기준으로 K개의 군집으로 나누는 비지도 학습 알고리즘.
- 대표적인 거리 기반 클러스터링 기법.

 구성 요소 순서 및 설명

1. **Select Columns in Dataset**
2. **Clean Missing Data**
3. **Normalize Data**
 - 거리 기반 알고리즘 특성상 스케일 정규화 필요 (0~1 범위)
4. **K-means Clustering**
 - 클러스터 수 K 지정 (예: K=3), 군집화 수행
 - 파라미터:
 - 초기 중심값 지정 방식: Random
 - 반복 횟수 설정 가능
5. **Assign Data to Clusters**
 - 각 샘플에 클러스터 번호 부여
6. **Evaluate Model (Optional)**
 - 군집별 통계 요약 등 확인 (별도 지표 없음)

모델 평가 지표

K-means는 지도 학습이 아니므로 일반적인 Accuracy, F1 등은 사용되지 않음

- Within-cluster SSE (Sum of Squared Error): 각 클러스터 내부 분산. 작을수록 좋음
- Silhouette Score (외부 계산 필요): 클러스터 응집도 및 분리도 평가

K-means Clustering + 로지스틱 회귀

구성 요소 순서 및 설명

1. **Select Columns in Dataset**
2. **Normalize Data**
3. **K-means Clustering**
 - k=3 (클러스터 수)
 - 거리: 유클리디언
4. **Assign Data to Clusters**
5. **Edit Metadata**
6. **Split Data**
7. **Multiclass Logistic Regression**
8. **Score Model** → **Evaluate Model**

모델 평가 지표

- Accuracy: 예측 정확도 (클래스 분류 정확성)
- 로지스틱 회귀 결과로 측정

분류 모델

랜덤 포레스트 (Two-Class Decision Forest)

- 여러 결정 트리를 랜덤하게 훈련한 후 그 결과를 평균내어 예측을 수행하는 앙상블 분류 알고리즘
- 과적합에 강하고 분류 정확도가 높음

✅ 구성 요소 순서 및 설명

- | | |
|---|--|
| 1. Select Columns in Dataset
→ 분석에 필요한 컬럼만 추출 | 6. Two-Class Decision Forest
→ 앙상블 기반 이진 분류 모델 |
| 2. Edit Metadata
→ 범주형 변수 지정 | 7. Train Model
→ 학습 수행 |
| 4. Convert to Indicator Values
→ 범주형 변수를 One-Hot Encoding | 8. Score Model
→ 테스트 데이터 예측 |
| 5. Split Data
→ 학습/테스트 데이터 분리 7:3 | 9. Evaluate Model
→ 분류 성능 평가 |

📊 모델 평가 지표

- Accuracy: 전체 중 맞춘 비율 (높을수록 좋음)
- Precision: Positive로 예측한 것 중 실제 Positive
- Recall: 실제 Positive 중 맞춘 비율
- F1 Score: Precision과 Recall의 조화 평균. 두 지표의 균형
- AUC: 0.5~1 사이, 1에 가까울수록 좋음

Two-Class Random Forest

- 여러 개의 결정 트리를 랜덤하게 생성하고 투표 방식을 통해 최종 결과를 도출하는 앙상블 기반 이진 분류 모델.

- 과적합에 강하고 성능이 안정적임.

✓ 구성 요소 순서 및 설명

1. **Select Columns in Dataset**
2. **Edit Metadata**
3. **Clean Missing Data**
4. **Convert to Indicator Values**
5. **Split Data**
6. **Two-Class Decision Forest**
→ 이진 분류용 앙상블 모델
 - 트리 수: 100
 - 최대 깊이, 샘플 수 등 기본값 사용
7. **Train Model** → **Score Model** → **Evaluate Model**

📊 모델 평가 지표

- Accuracy: 전체 중 정답률. 높을수록 좋음
- Precision: Positive로 예측한 것 중 실제 Positive 비율
- Recall: 실제 Positive 중 올바르게 예측한 비율
- F1 Score: Precision과 Recall의 조화 평균
- AUC: 0.5 ~ 1 사이. 1에 가까울수록 분류 우수

Two-Class Boosted Decision Tree + 튜닝

- 반복적으로 약한 결정 트리를 쌓아가며 오차를 보완해가는 방식의 앙상블 분류 알고리즘.
- 예측 성능이 높고 불균형 데이터에 강함.

✓ 구성 요소 순서 및 설명

1. **Select Columns in Dataset**
2. **Edit Metadata**
3. **Clean Missing Data**
4. **Convert to Indicator Values**
5. **Split Data**
6. **Two-Class Boosted Decision Tree**
→ 앙상블 분류 모델(Gradient Boosting 기반)
6. **Train Model**
7. **Tune Model Hyperparameters**
→ Boosted Tree에 대해 자동으로 최적 하이퍼파라미터 탐색
→ 사용된 설정:
 - 탐색 전략: Random sweep
 - 평가 지표: Accuracy
8. **Score Model** → **Evaluate Model**

모델 평가 지표

- Accuracy: 전체 예측 중 정답 비율. 높을수록 좋음
 - Precision: Positive 예측 중 실제 Positive 비율
 - Recall: 실제 Positive 중 모델이 맞춘 비율
 - F1 Score: Precision과 Recall의 조화 평균
 - AUC: 분류 모델의 예측력 종합 지표. 0.5~1 범위
-

Two-Class Decision Forest + 카이제곱 피처 선택

- 랜덤 포레스트를 사용한 이진 분류 모델이며, 그 이전에 카이제곱 통계를 활용해 중요 피처만 선택하여 학습 성능을 높이는 절차를 포함함.

구성 요소 순서 및 설명

1. **Select Columns in Dataset**
2. **Edit Metadata**
3. **Clean Missing Data**
4. **Convert to Indicator Values**
5. **Filter Based Feature Selection**
 - Method: 카이제곱 (Chi-Squared) 통계
 - Target column: 예측할 이진 타겟
 - Number of features: 원하는 개수 설정 (예: 상위 8개)
6. **Split Data**
7. **Two-Class Decision Forest**
→ 카이제곱으로 선정된 피처 기반 분류 모델
8. **Train Model** → **Score Model** → **Evaluate Model**

모델 평가 지표

- Accuracy: 전체 정확도. 높을수록 좋음
 - Precision: Positive 예측 중 실제 Positive 비율
 - Recall: 실제 Positive 중 모델이 올바르게 예측한 비율
 - F1 Score: Precision과 Recall의 균형
 - AUC: 분류기 전체 성능 종합. 1에 가까울수록 우수
-

Two-Class Support Vector Machine (SVM)

- 데이터를 고차원 공간으로 매핑한 뒤, 클래스 간의 마진이 가장 넓은 초평면을 찾아 분류하는 알고리즘
- 고차원 분리 문제에 특히 효과적이며, 커널 함수를 사용하여 비선형 문제에도 대응할 수 있음

✅ 구성 요소 순서 및 설명

1. **Select Columns in Dataset**
2. **Edit Metadata**
3. **Clean Missing Data**
4. **Convert to Indicator Values**
5. **Normalize Data**
→ SVM은 거리 기반 모델이므로 정규화가 매우 중요함(tanh 사용)
6. **Execute Python Script**
→ 비용 민감 처리
7. **Split Data**
7. **Two-Class Support Vector Machine**
→ SVM 분류 모델 적용
 - 커널 종류: 기본값 (선형 or RBF, 명시 X)
 - 복잡도(C): 기본값 사용
 - Number of iterations: 100 (기본값)
8. **Train Model** → **Score Model** → **Evaluate Model**

📊 모델 평가 지표

- Accuracy: 전체 예측 중 정답 비율
- Precision: Positive로 예측한 것 중 실제 Positive 비율
- Recall: 실제 Positive 중 모델이 맞춘 비율
- F1 Score: Precision과 Recall의 조화 평균
- AUC: 전체 분류 성능 종합 지표

```
# Execute Python Script: 비용 민감 처리
# 소수 클래스(고위험 그룹)를 인위적으로 복제하여 불균형 보정
import pandas as pd

def azureml_main(dataframe1 = None, dataframe2 = None):
    # iloc[:, 7]: 레이블 컬럼 (신용등급 or 위험도 등)이 1 or 2인 행 필터링
    df_label_1 = dataframe1[dataframe1.iloc[:, 7] == 1] # 저위험
    df_label_2 = dataframe1[dataframe1.iloc[:, 7] == 2] # 고위험

    # 고위험 데이터를 5배 복제하여 부풀리기(
    # SMOTE 대신 직접 복제 방식을 사용한 oversampling 기법
```

```
result = df_label_1.append([df_label_2] * 5, ignore_index=True)

return result,
```

Two-Class Logistic Regression + Stratified Split

- 로지스틱 회귀는 확률을 기반으로 이진 분류를 수행하는 선형 모델
- 입력값에 sigmoid 함수를 적용해 클래스 0 또는 1로 예측
- **층화추출(stratified split)**은 불균형 데이터를 공정하게 나누기 위해 각 클래스 비율을 유지한 채 학습/테스트셋 분할

✅ 구성 요소 순서 및 설명

1. [Select Columns in Dataset](#)
2. [Edit Metadata](#)
3. [Clean Missing Data](#)
4. [Convert to Indicator Values](#)
5. [Split Data](#)
 - Stratified split 옵션을 True로 설정
 - 클래스 불균형 문제 완화
6. [Two-Class Logistic Regression](#)
7. [Score Model](#) → [Evaluate Model](#)

📊 모델 평가 지표

- Accuracy: 전체 예측 중 맞춘 비율 (높을수록 좋음)
- Precision: 양성으로 예측한 것 중 실제 양성 비율
- Recall: 실제 양성 중 맞춘 비율
- F1 Score: 정밀도와 재현율의 조화 평균
- AUC: 분류기의 종합적인 성능 (1에 가까울수록 좋음)

Two-Class Logistic Regression + Cross Validation

- 로지스틱 회귀는 이진 분류를 위한 선형 모델로, 각 입력에 대해 특정 클래스일 확률을 예측
- **교차검증(Cross Validation)**은 모델의 일반화 성능을 평가하기 위한 방법으로, 데이터를 여러 조각으로 나누어 반복적으로 학습/테스트를 수행함

✅ 구성 요소 순서 및 설명

1. `Select Columns in Dataset`
2. `Edit Metadata`
3. `Clean Missing Data`
4. `Convert to Indicator Values`
5. `Two-Class Logistic Regression`
6. `Cross-Validate Model`
 - Fold 수 설정 (예: 5-Fold)
 - 모델의 일반화 성능을 보다 정확하게 평가
7. `Evaluate Model`

모델 평가 지표

- Accuracy: 전체 예측 중 정답 비율
- Precision: Positive로 예측한 것 중 실제 Positive 비율
- Recall: 실제 Positive 중 맞춘 비율
- F1 Score: Precision과 Recall의 조화 평균
- AUC: 모델 분류 성능의 종합 평가 지표

Boosted Decision Tree + SMOTE

- 여러 개의 약한 트리를 순차적으로 학습시켜 오차를 보정하는 방식의 강력한 앙상블 분류기
- **SMOTE** (Synthetic Minority Over-sampling Technique)는 소수 클래스의 데이터를 **합성하여** 클래스 불균형 문제를 해결하는 전처리 기법

구성 요소 순서 및 설명

1. `Select Columns in Dataset`
2. `Execute Python Script 1`
3. `Execute Python Script 2`
4. `Edit Metadata`
5. `Clean Missing Data`
6. `Convert to Indicator Values`
7. `SMOTE`
 - 소수 클래스 데이터를 합성하여 클래스 균형 조정
 - Oversampling rate: 기본값
 - k-nearest neighbors: 기본값
6. `Split Data`
7. `Two-Class Boosted Decision Tree`
 - Max number of leaves per tree
 - Learning rate
 - Minimum number of samples per leaf
8. `Train Model` → `Score Model` → `Evaluate Model`

모델 평가 지표

- Accuracy: 전체 예측 중 정답 비율
- Precision: Positive로 예측한 것 중 실제 Positive 비율
- Recall: 실제 Positive 중 모델이 맞춘 비율
- F1 Score: Precision과 Recall의 조화 평균
- AUC: 모델 분류 성능의 종합 지표 (1에 가까울수록 우수)

```
# Execute Python Script 1: 결측값 많은 행 제거
def azureml_main(dataframe1 = None, dataframe2 = None):
    rate = 0.7
    nan_ratio_per_row = dataframe1.isna().mean(axis=1)
    df_cleaned = dataframe1[nan_ratio_per_row < rate]
    return df_cleaned,

# Execute Python Script 2: 0이 많은 열 제거
def azureml_main(dataframe1 = None, dataframe2 = None):
    rate = 0.7
    zero_ratio_cols = (dataframe1 == 0).mean()
    df_cleaned = dataframe1.drop(columns = zero_ratio_cols[zero_ratio_cols >= rate].index)
    return df_cleaned,
```

회귀 모델

선형 회귀 (Linear Regression)

- 입력 변수와 출력 변수 간 선형 관계를 기반으로 수치 값을 예측하는 회귀 알고리즘.
- 빠르고 해석이 쉬움.

구성 요소 순서 및 설명

1. Execute Python Script

→ 분석에 불필요한 컬럼 제거

```
import pandas as pd

def azureml_main(dataframe1 = None, dataframe2 = None):
    print("자전거 렌탈 데이터:")
```

```
print(dataframe1.head())

# 불필요한 컬럼 제거
df = dataframe1.drop(columns=['instant', 'dteday', 'casual', 'registered'])

return df,
```

2. [Select Columns in Dataset](#)
3. [Edit Metadata](#)
4. [Convert to Indicator Values](#)
5. [Normalize Data](#)
6. [Split Data](#)
7. [Linear Regression](#)
8. [Train Model](#) → [Score Model](#) → [Evaluate Model](#)

모델 평가 지표

- MAE (Mean Absolute Error): 예측 오차의 절댓값 평균. 작을수록 좋음
- MSE (Mean Squared Error): 오차 제곱 평균. 이상치에 민감, 작을수록 좋음
- RMSE: MSE의 제곱근. 직관적 해석 가능

Decision Tree Regression

- 데이터를 여러 조건에 따라 트리 구조로 분할해가며 예측값을 계산하는 결정 트리 기반 회귀 모델.
- 비선형 관계나 복잡한 조건에서도 유연하게 대응 가능.

구성 요소 순서 및 설명

1. [Select Columns in Dataset](#)
2. [Clean Missing Data](#)
3. [Edit Metadata](#)
4. [Convert to Indicator Values](#)
5. [Split Data](#)
6. [Decision Forest Regression](#)
→ 회귀 트리 모델 구성
 - 반복적으로 분할하며 예측
 - 분기 기준: 최소 잔차 제곱합(SSE)
7. [Train Model](#) → [Score Model](#) → [Evaluate Model](#)

모델 평가 지표

- MAE (Mean Absolute Error): 예측값과 실제값의 절대 차이 평균. 작을수록 좋음
- MSE (Mean Squared Error): 오차 제곱의 평균. 이상치에 민감

- RMSE (Root Mean Squared Error): MSE의 제곱근. 오차를 실제 단위로 해석 가능

시각화 Code

Heatmap

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from azureml.core import Run

def azureml_main(dataframe1 = None, dataframe2 = None):
    plt.figure(figsize=(10, 8))
    corr_matrix = dataframe1.corr()
    sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm")
    plt.title("Heatmap")

    img_file = "heatmap.png"
    plt.savefig(img_file)
    plt.close()

    run = Run.get_context(allow_offline=True)
    run.upload_file(f"graphics/{img_file}", img_file)

    return dataframe1,
```

Scatterplot

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from azureml.core import Run

def azureml_main(dataframe1 = None, dataframe2 = None):

    plt.figure(figsize=(10, 8))
    sns.scatterplot(data=dataframe1, x='bill_length_mm', y='bill_depth_mm', hue='species')
    plt.title('Bill length and depth by species')
    plt.legend(title='species')
```

```
img_file = "scatterplot.png"
plt.savefig(img_file)
plt.close()

run = Run.get_context(allow_offline=True)
run.upload_file(f"graphics/{img_file}", img_file)

return dataframe1,
```

Normalize Transformation Method 정리

방법	설명	사용하면 좋은 상황
ZScore	평균 0, 표준편차 1로 표준화 $x' = \frac{x - \mu}{\sigma}$	- 데이터가 정규분포(또는 근사)일 때 - 선형 회귀, 로지스틱 회귀, SVM 등 거리 기반 모델
MinMax	모든 값을 0~1로 정규화 $x' = \frac{x - \min}{\max - \min}$	- 트리 계열 모델 (Decision Tree, Random Forest 등)은 영향 없음 - K-means, 신경망 등 값의 범위 중요할 때
Logistic	시그모이드 함수 사용 $x' = \frac{1}{1 + e^{-x}}$	- 데이터가 극단적으로 퍼져 있고 - 0~1 사이 확률형태로 변환하고 싶을 때
LogNormal	로그-정규 변환 $x' = \ln(x + 1)$	- 양수 데이터이고 - 극단적인 이상치, 오른쪽 꼬리가 긴 분포에서 사용
Tanh	쌍곡 탄젠트 함수 사용 $x' = \tanh(x)$	- 평균 0 중심으로 압축하고 싶을 때 - 분포를 [-1, 1]로 제한하고 싶을 때 (신경망 입력 등)

모델별 추천 방식

모델	추천 정규화 방식
선형 회귀 / 로지스틱 회귀	ZScore
SVM	ZScore
K-means	MinMax or ZScore
신경망 (Neural Network)	MinMax, Tanh, Logistic

모델	추천 정규화 방식
트리 계열 모델 (DT, RF, Boosted DT)	❌ 필요 없음 (트리는 정규화에 영향 거의 없음)
PCA, 거리 기반 알고리즘	ZScore or MinMax

- ZScore vs MinMax 선택 기준
 - 이상치에 민감하면 ZScore,
 - 값을 모두 0~1로 고정하고 싶으면 MinMax
- Log/LogNormal은 값이 양수일 때만 사용하세요. 음수 있으면 오류 발생합니다.